Copyright

Warning
This material has been reproduced and communicated to you by or on behalf of Federation University Australia in accordance with Section 113P of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material may be the subject of copyright protection under the Act.

Do not remove this notice.

ITECH3102 –Data Visualization
Week 3

Lab Exercises –JavaScript

ID: _____Name: _____

In this week's projects, we discuss some fundamentals of JavaScript. We also introduce two approaches to create and edit JavaScript: using Notepad and using Developer tools.

Project 1: JavaScript

JavaScript is the scripting language that can make pages dynamic by manipulating the DOM after a page has already loaded in the browser. As I mentioned earlier, getting to know D3 is also a process of getting to know JavaScript. We'll dig in deeper as we go, but here is a taste to get you started.

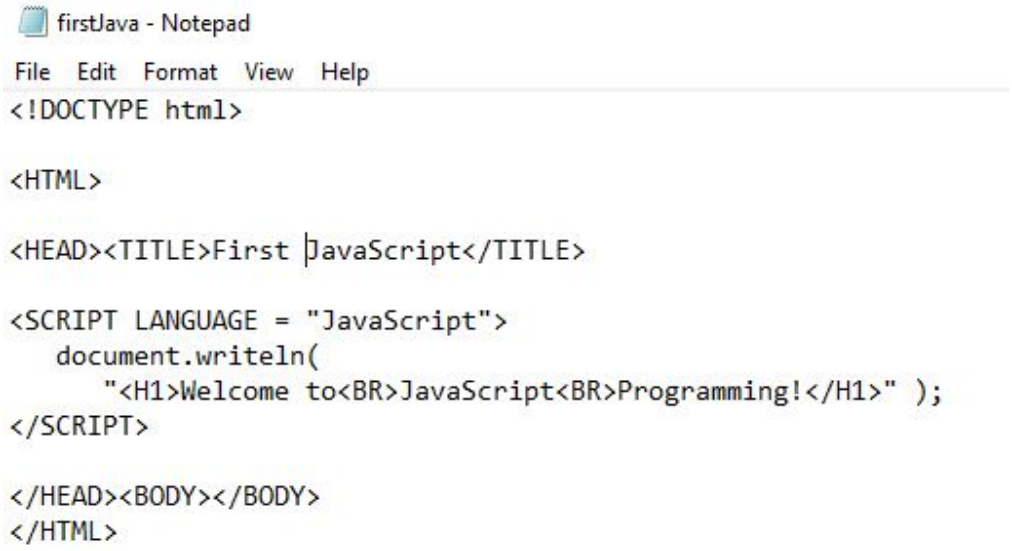
Hello, Console
Normally, we write JavaScript code (or, a "script") in a text file, and then load that file to the browser in a web page. But you can also just type JavaScript code directly into your browser. This is an easy and quick way to get your feet wet and test out code. We'll also use the JavaScript console for debugging, as it's an essential tool for seeing what's going on with your code.

1. Using Notepad to create JavaScript

Normally, we write JavaScript code (or, a “script”) in a text file, and then load that file to the browser in a web page.

Type the following codes

Open Notepad from Windows Accessories panel in your computer and type the following codes.

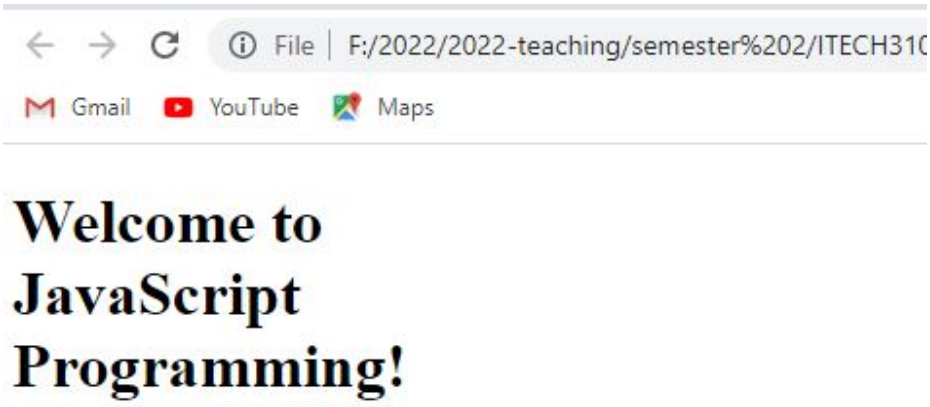


Save your HTML file

To save above codes as HTML file, you need to Select the File Menu from the Top Navigation Bar in Notepad Text Editor and then click 'Save As', type “firstJava.html” in the box of File name. Here firstJava is the name of your file and .html is extension.

View the firstJava.html page in your web browser

After saving firstJava.html in your computer, right click the html file and select Google Chrome. You’ll see the following:



We explain the above codes later.

2. Using browser's built-in developer tools

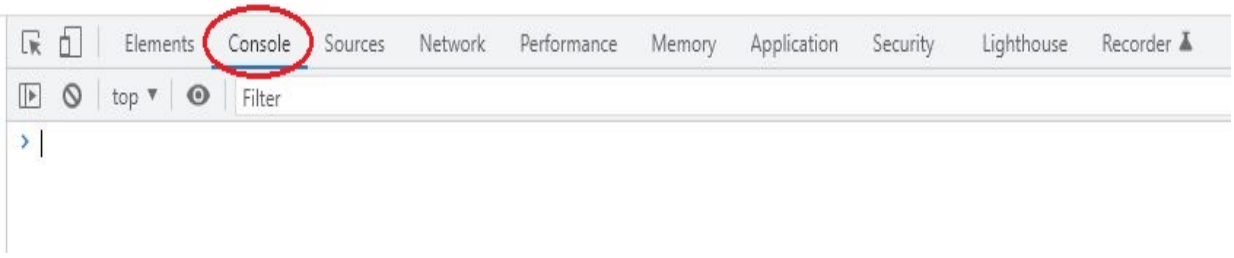
You can also just type JavaScript code directly into your browser. This is an easy and quick way to get your feet wet and test out code. In addition, you'll also use the JavaScript console for debugging, as it's an essential tool for seeing what's going on with your code.

In this course, we use Google Chrome browser to view and edit JavaScript. However, you can also use other browsers to debug JavaScript code, such as using Microsoft Edge, etc.

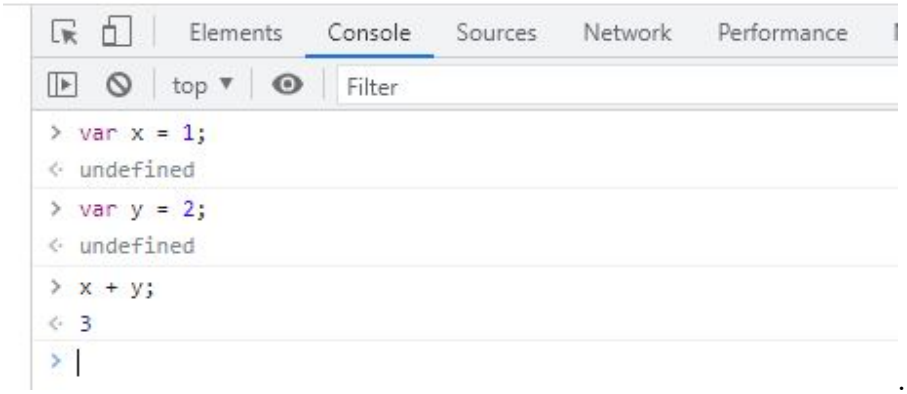
Directly write and test code by using Console provided by Chrome

(1) Open Developer tools

Click on the Chrome menu icon on the browser toolbar at the top right. This icon is a box with three vertical dots. Click More tools and click Developer tools to open it.
Then click Console to open it:



(2) Type the following codes in Console:



Then type the following codes in Console:

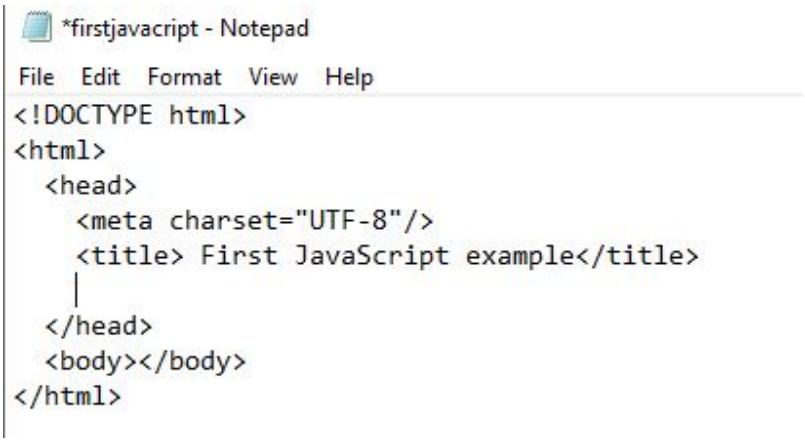
```
> console.log("Welcome to JavaScript Programming !");
  Welcome to JavaScript Programming !
```

In above codes, the console accepts one line of code at a time, and it always spits back the result of whatever you input. For example, if you enter x + y, the console returns the result of 3, which is a quick way to test out code.

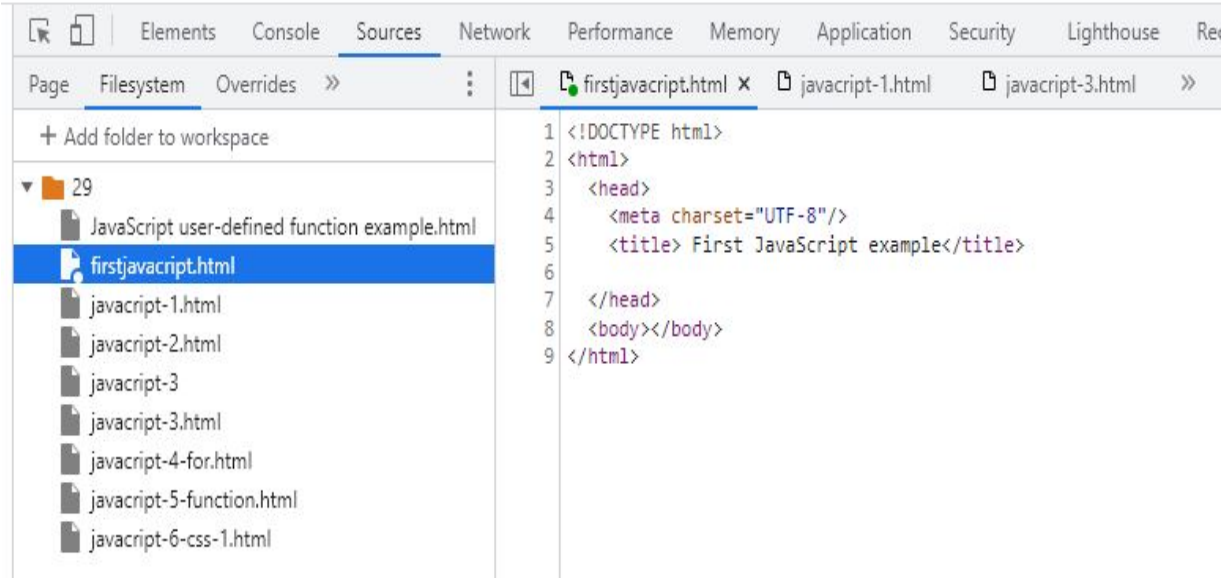
Other times, you'll want your script to print values out to the console automatically, so you can keep an eye on what's going on. As you can see in the example above, you can use console.log("something"); to do that. For example, console.log ("Welcome to JavaScript Programming!") returns Welcome to JavaScript Programming!

Write JavaScript code in a text file and edit the code in browser

Type the following codes in Notepad, save it as the html file:



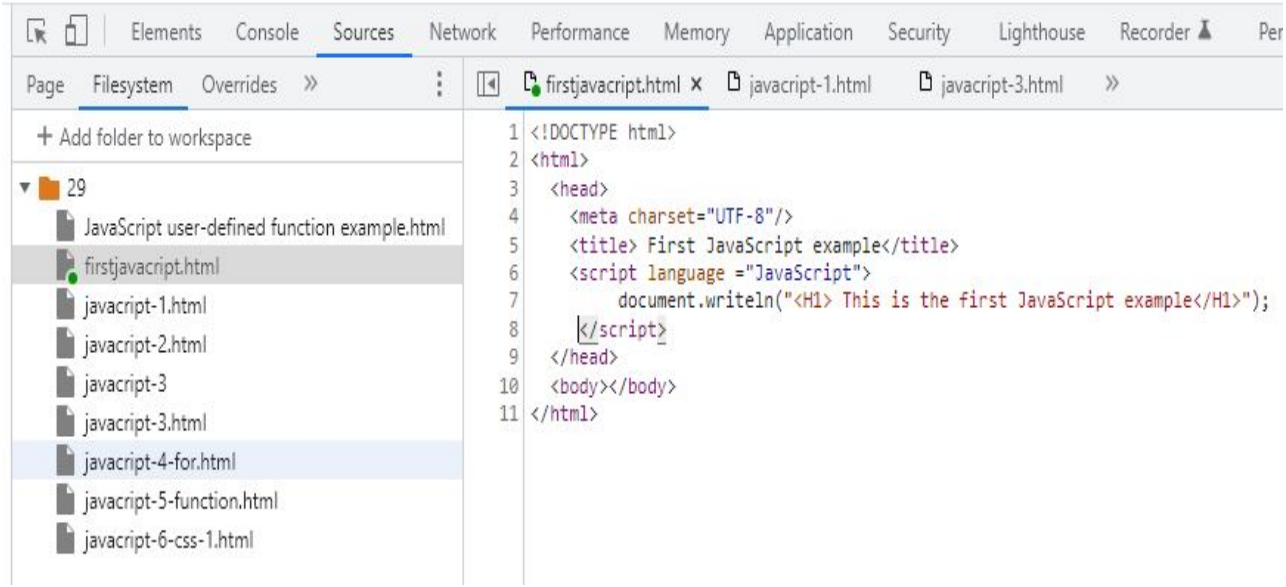
Then open it in Chrome. As there are no texts between <body> and </body>, there is nothing shown in browser.



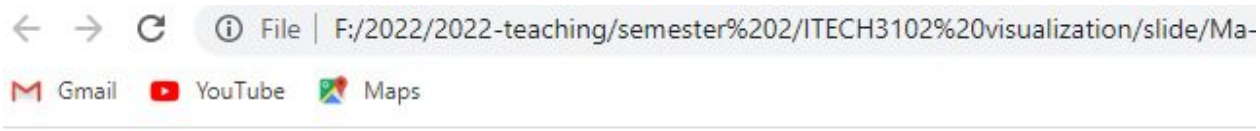
Edit and Save html file.

Add the following codes to the html file and then press **Ctrl+S** key to save the html file.

```
<script language ="JavaScript">
        document.writeln("<H1> This is the first JavaScript example</H1>");
</script>
```



After freshing your browser, you'll see the following texts displayed in broswer.



This is the first JavaScript example

In the above codes, we write scripts between <head> tag and </head> tag. In the future examples, we can also put scripts between <body> tag and </body> tag.

Line 6
<script language ="JavaScript">

uses the <script > tag to indicate to the browser that the text that follows is part of a script. The language attribute specifies the scripting language used in the script, in this case, JavaScript.

Line 7
document.writeln("<H1> This is the first JavaScript example</H1>");

instructs the browser's JavaScript interpreter to perform an action, namely to display in the Web page the string of characters contained between the double quotation (") marks. Line 7 uses the browser's **document** object, which represents the HTML document currently being displayed in the browser.

The document object's **writeln** method writes a line of HTML text in the HTML document being displayed.

Project 2: Variables, Arrays, Objects

1. Variables

Variables are containers for data. A simple variable holds one value:

var number = 5;

In that statement, var indicates you are declaring a new variable, the name of which is number. The equal's sign is an assignment operator because it takes the value on the right (5) and assigns it to the variable on the left (number).

2. Arrays

An array is a sequence of values, conveniently stored in a single variable.
Hard brackets [] indicate an array, and each value is separated by a comma:

var numbers = [5, 10, 15, 20, 25];

You can access (retrieve) a value in an array by using bracket notation:

numbers [2] //Returns 15

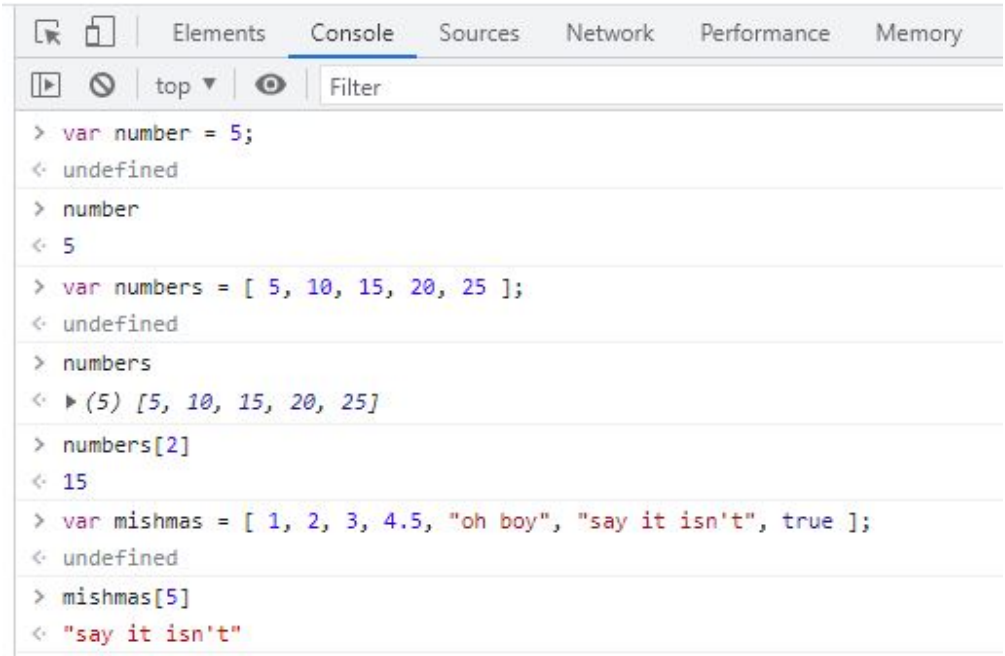
Arrays can contain any type of data, not just integers:

var percentages = [0.55, 0.32, 0.91];
var names = ["Ernie", "Bert", "Oscar”];

percentages [1] //Returns 0.32
names [1] //Returns "Bert"

var mishmash = [1, 2, 3, 4.5, 5.6, "oh boy", "say it isn't", true];

Type the following codes in browser’s Console:



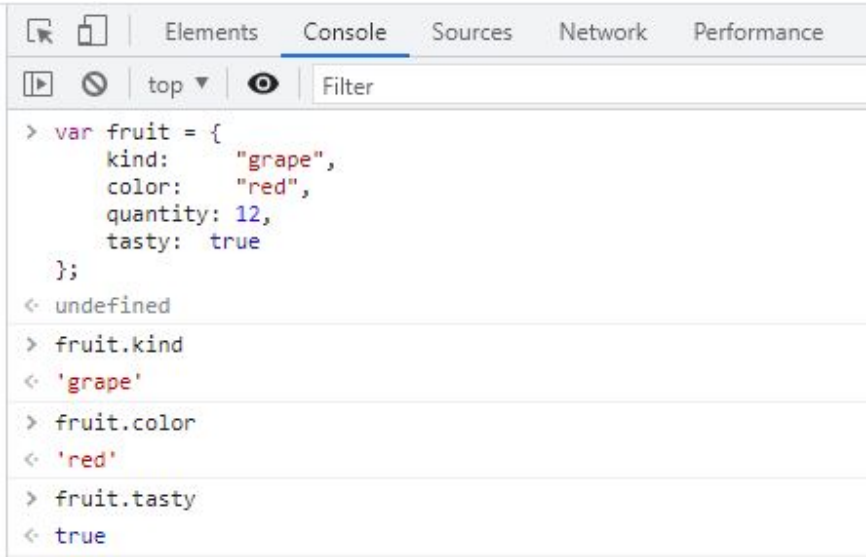
3. Objects

We can think of a JavaScript object as a custom data structure. We use curly brackets {} to indicate an object. In between the brackets, we include properties and values. A colon: separates each property and its value, and a comma separates each property/value pair:

```
var fruit = {
   kind: "grape",
   color: "red",
   quantity: 12,
   tasty: true
};
```

To reference each value, we use dot notation, specifying the name of the property:

Type the following codes in browser's Console:



4. Objects and Arrays

You can combine these two structures to create arrays of objects, or objects of arrays, or objects of objects or, well, basically whatever structure makes sense for your dataset.

We use hard brackets [ ] on the outside, to indicate an array, followed by curly brackets {} and object notation on the inside, with each object separated by a comma:

```
var fruits = [
```

```
    {
      kind: "grape",
      color: "red",
      quantity: 12,
      tasty: true
    },
    {
      kind: "kiwi",
      color: "brown",
      quantity: 98,
      tasty: true
    },
    {
      kind: "banana",
      color: "yellow",
      quantity: 0,
      tasty: true
    }
];
```

To access this data, we just follow the trail of properties down to the values we want. Remember, [ ] means array, and {} means object. fruits is an array, so first we use bracket notation to specify an array index:

Type the following codes in browser's Console:

```
> var fruits =[
      {
          kind: "grape",
          color: "red",
          quantity: 12,
          tasty: true
      },
      {
          kind: "kiwi",
          color: "brown",
          quantity: 98,
          tasty: true
      },
      {
          kind: "banana",
          color: "hellow",
          quantity: 0,
          tasty: true

      }
  ];
<- undefined
> fruits[1].quantity
<- 98
> fruits[0].kind
<- 'grape'
> fruits[1].kind
<- 'kiwi'
> fruits[2].kind
<- 'banana'
```

5. JSON

You will encounter JavaScript Object Notation (JSON). You can read up on the details, but JSON is basically a specific syntax for organizing data as JavaScript objects. The syntax is optimized for use with JavaScript (obviously) and AJAX requests, which is why you'll see a lot of web-based application programming interfaces (APIs) that return data formatted as JSON. It's faster and easier to parse with JavaScript than XML, and of course D3 works well with it.

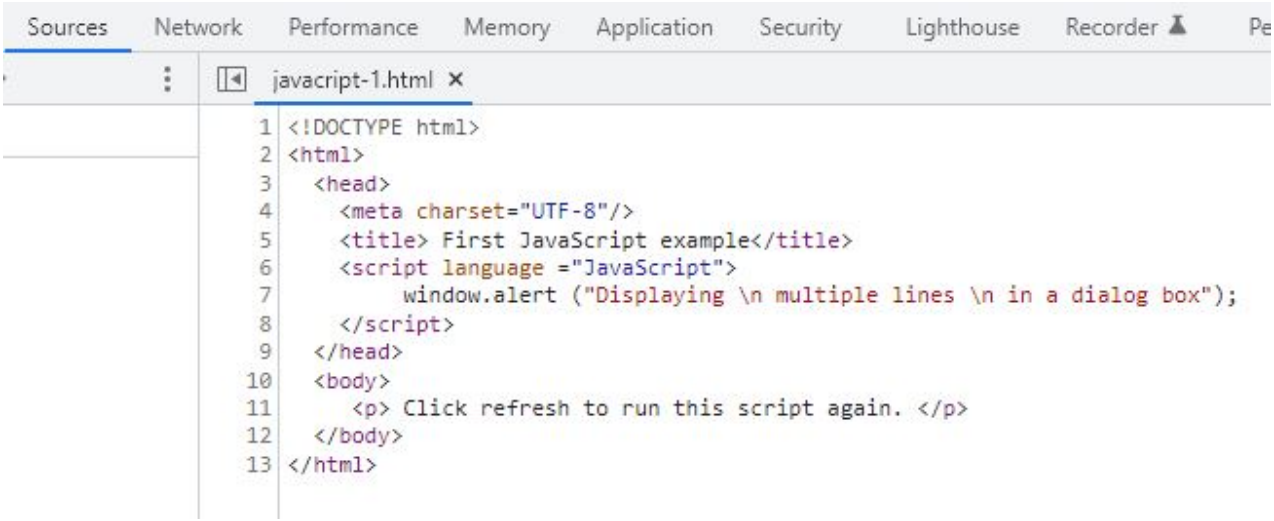
All that, and it doesn't look much weirder than what we've already seen:

```
{
  "kind": "grape",
  "color": "red",
  "quantity": 12,
  "tasty": true
}
```

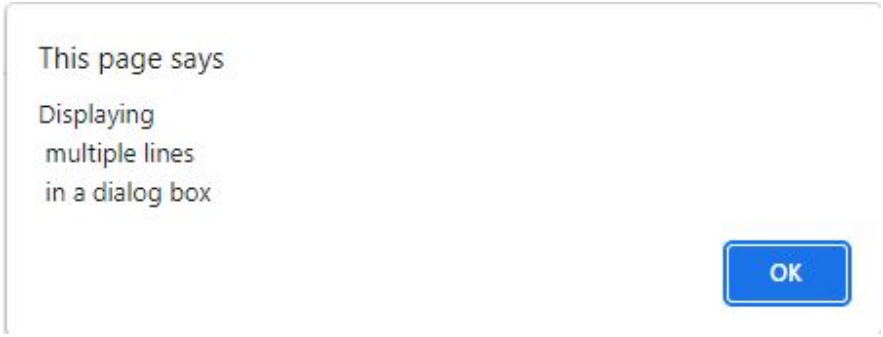
The only difference here is that our property names are now surrounded by double quotation marks "", making them string values.

Examples

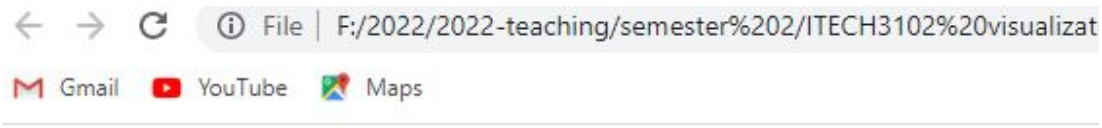
Type the following codes in text editor like Notepad, save it as javacript-1.html and open it by using Chrome:



And you'll get:



After clicking Ok button, you'll get the following text in browser:



Click refresh to run this script again.

In the above codes, line 7
window.alert ("Displaying \n multiple lines \n in a dialog box");

uses the browser's window object to display an alert dialog box. Method alert of the window object requires as its argument the string to display.
The escape sequence \n is the newline character, which causes the cursor to move to the beginning of the next line in the dialog box.

Project 3: Basic control structures of JavaScript

Whenever your code needs to make a decision or repeat something, you need a control structure. There are lots to choose from, but we are primarily interested in if statements and for loops and while loops.

1. **Selection structure**

JavaScript provides selection structures, such as if selection structure, the if/else selection structure and switch selection structure, etc.

if () only

An if statement uses comparison operators to determine if a statement is true or false:

```
if (test) {
   //Code to run if true
}
```

If the test between parentheses is true, then the code between the curly brackets is run. If the test turns up false, then the bracketed code is ignored, and life goes on. (Technically, life goes on either way.)

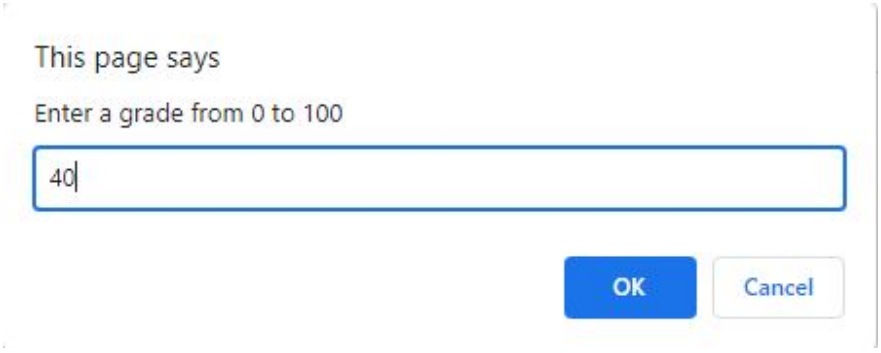
Type the following codes in text editor like Notepad, and save it.

*javacript-2 - Notepad

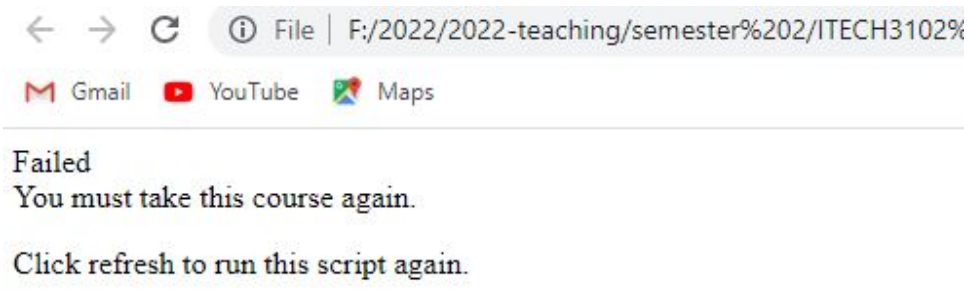
File Edit Format View Help

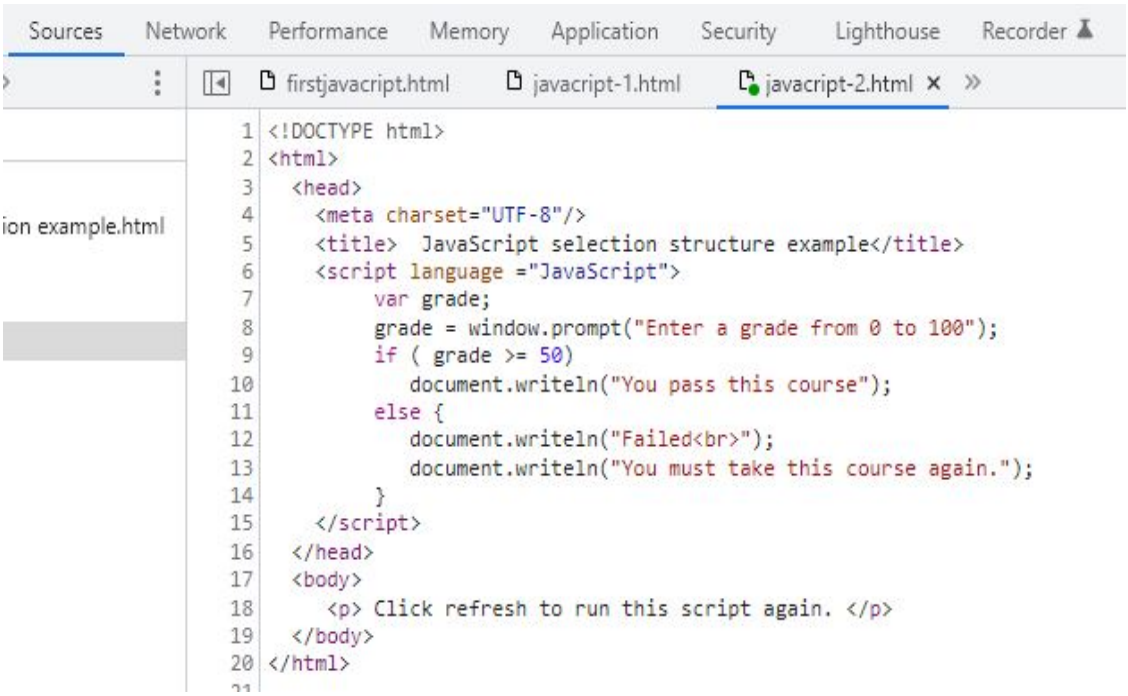
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>  JavaScript selection structure example</title>
    <script language ="JavaScript">
         var grade;
         grade = window.prompt("Enter a grade from 0 to 100");
         if ( grade >= 50)
            document.writeln("You pass this course");
         else {
            document.writeln("Failed<br>");
            document.writeln("You must take this course again.");
         }
    </script>
  </head>
  <body>
     <p> Click refresh to run this script again. </p>
  </body>
</html>
```

After saving the html file and openng it by using a browser, you'll get:



Type 40 and press Ok button, and you'll get:



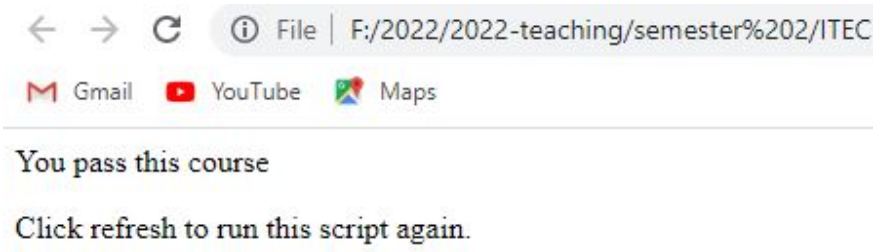


In the above codes, line 7
var grade

declares the variable grade used in the script.

Line 8
grade = window.prompt("Enter a grade from 0 to 100");

uses window.promot to allow the user to input a grade value and store it in variable grade. If grade is greater than or equal to 50, the program executes line 10 and then ignores statements within the body of else. It displays:



Otherwise, if grade is less than 50, the program executes both statements in the body of the else and prints:

Failed
You must take this course again.

2 Repetition structures

JavaScript also provides repetition structures, such as while, do/while, for and for/in structures, etc.

(1) for () now

You can use for loops to repeatedly execute the same code, with slight variations.
A for loop uses this syntax:

```
for (initialization; test; update) {
   //Code to run each time through the loop
}
```

They are so called because they loop through the code *for* as many times as specified. First, the initialization statement is run. Then, the test is evaluated, like a tiny if statement. If the test is true, then the bracketed code is run. Finally, the update statement is run, and the test is revaluated.

The most common application of a for loop is to increase some variable by 1 each time through the loop. The test statement can then control how many times the loop is run by referencing that value. (The variable is often named i for index, purely by convention, because it is short and easy to type.)

```
for (var i = 0; i < 5; i++) {
   console.log(i);  //Prints value to console
}
```

The preceding for loop prints the following to the console:

```
0

1

2

3

4
```

The first time through, a variable named `i` is declared and set to zero. `i` is less than five, so the bracketed code is run, printing the current value of `i` (zero) to the console. Last, the value of `i` is increased by one. (`i++` is shorthand for `i = i + 1`.) Now `i` is `1`, so the test again evaluates as true, and the bracketed code is run again, but this time the value printed to the console is `1`.

Please note that

- Pre-increment: x is incremented by 1, then y is assigned the value of x
  x = 1;
  y = ++x; // x is now 2, y is also 2

- // Post-increment: y is assigned the value of x, then x is incremented by 1
  x = 1;
  y = x++; // y is 1, x is now 2

Type the following codes in text editor like Notepad and save it.

javacript-4-for - Notepad

File Edit Format View Help

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>  JavaScript repetion structure example: for  </title>
    <script language ="JavaScript">

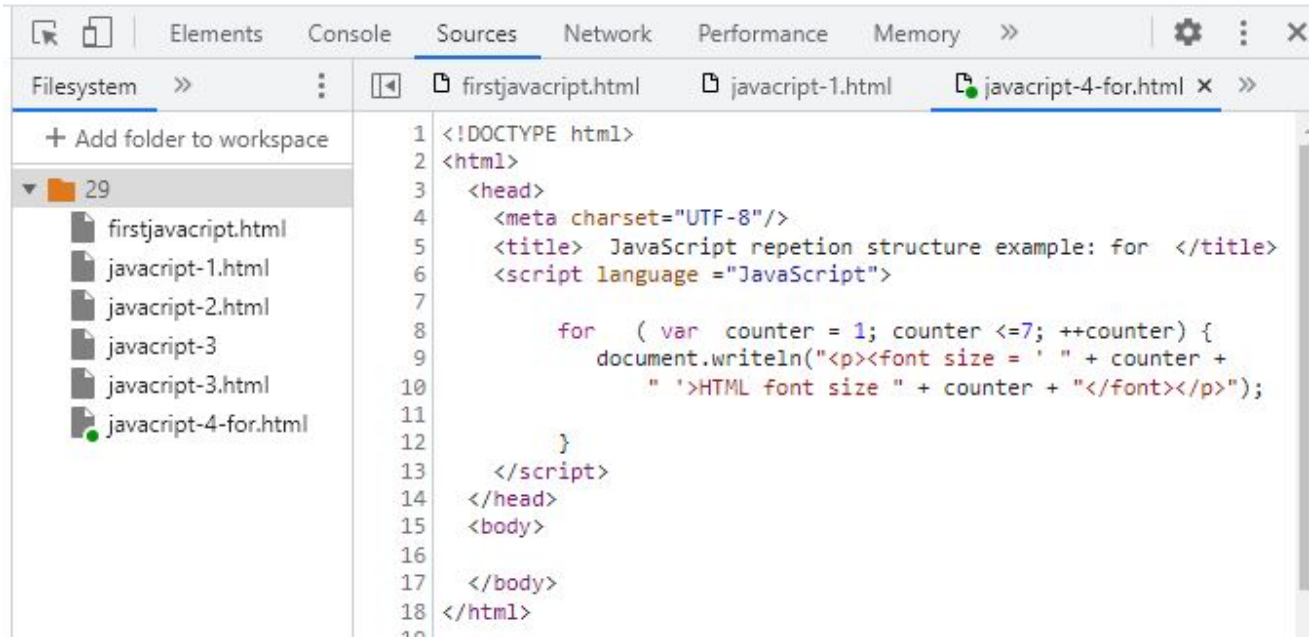
          for   ( var  counter = 1; counter <=7; ++counter) {
             document.writeln("<p><font size = ' " + counter +
                 " '>HTML font size " + counter + "</font></p>");

          }
    </script>
  </head>
  <body>

  </body>
</html>
```

After saving the html file and openng it by using a browser, you'll get:





In the above codes, line 8 provides a for repetition structure where var counter = 1 is loop's control variable and provides its initial value; counter <=7 is the expression of loop continuation test, which tests the loop-continuation condition; and ++counter is an expression of increment that increments the control variable.

(2) While ()

Type the following codes in text editor like Notepad.

javacript-3 - Notepad

File Edit Format View Help

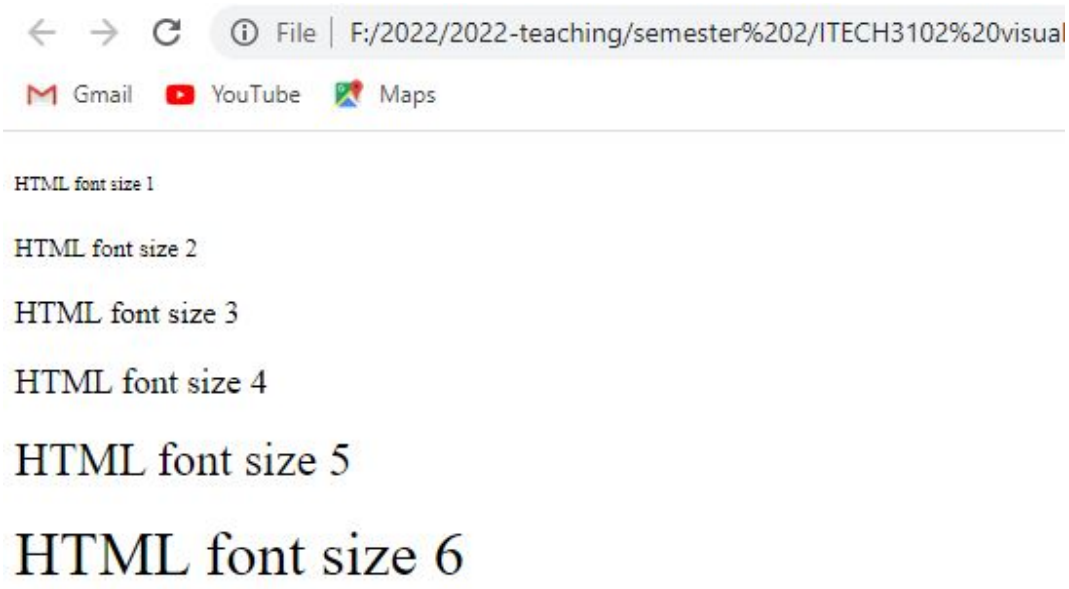
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>  JavaScript repetion structure example: while </title>
    <script language ="JavaScript">
         var counter = 1;              // initialization

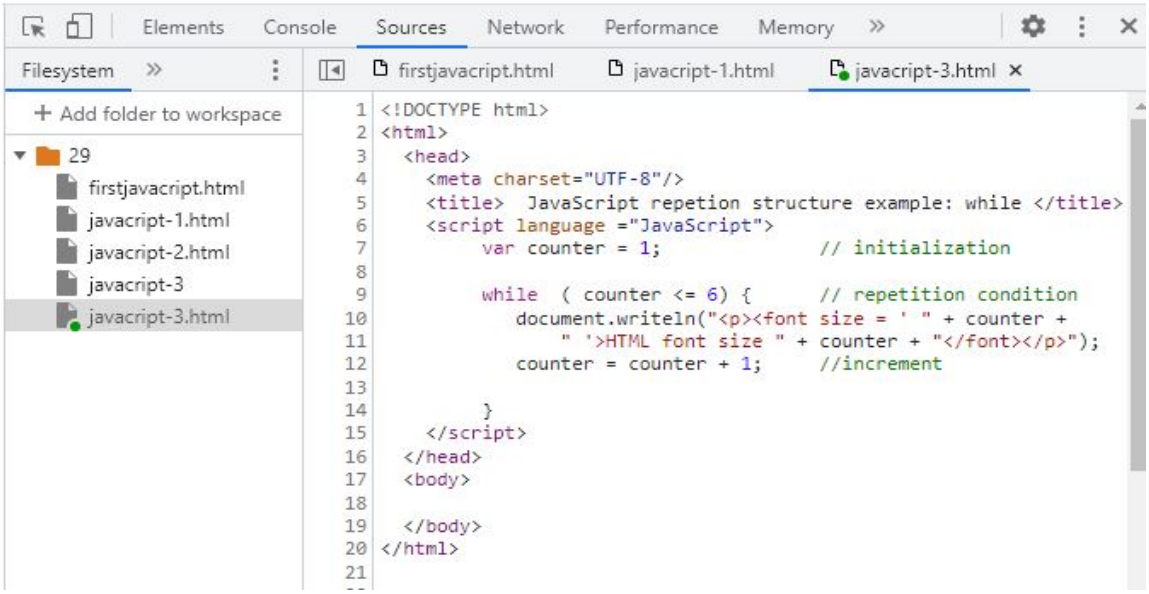
         while  ( counter <= 6) {      // repetition condition
            document.writeln("<p><font size = ' " + counter +
                " '>HTML font size " + counter + "</font></p>");
            counter = counter + 1;     //increment

         }
    </script>
  </head>
  <body>

  </body>
</html>
```

After saving the html file and openng it by using a browser, you'll get:





In the above codes, line 7

declares a control variable (counter) and sets it to an initial value of 1.

Lines 9, 10 and 11 in the while structure

```
document.writeln("<p><font size = ' " + counter +
            " '>HTML font size " + counter + "</font></p>");
```

write a paragraph consisting of the string “HTML font size”, which is concatenated with the control variable counter’s value by using +. Counter’s value represents the font size. Please note that we use the single quotes ‘ that are placed around the value of counter in the concatenated string.

Line 12

counter = counter + 1;

increments the control variable by 1 for each iteration of the loop (each time the body of the loop is performed).

3. Function

Each script consists of a series of statements and control structures in sequence. These scripts have been executed as the browser loads the Web page and evaluates the <head> section of the page. We now introduce how we can write our own customized functions and call them in a script.

Functions are chunks of code that do things.

More specifically, functions are special because they can take arguments or parameters as input, and then return values as output. Parentheses are used to *call* (execute) a function. If that function requires any arguments (input values), then you *pass* them to the function by including them in the parentheses.
Whenever you see something like the following code, you know it's a function:

```
calculateGratuity (38);
```

Please note that in the previous and current examples, we show that the <body> element of the HTML document is empty and the document is created directly by a JavaScript. But JavaScript can interact with the elements in the <body> of a document.

Type the following codes in text editor like Notepad.

javacript-5-function - Notepad

File Edit Format View Help

```
<!DOCTYPE html >
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>  JavaScript user-defined function example  </title>
    <script language ="JavaScript">
         document.writeln(
               "<H1>Square the numbers from 1 to 10 </H1>" );

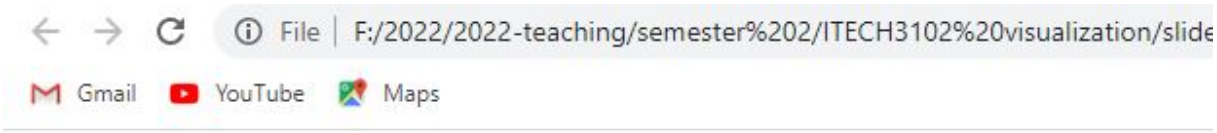
         for   ( var  x  = 1; x <= 10; ++x)
            document.writeln("The square of " + x + " is " +
                        square(x) + "<br>");

         function square(y)
         {
             return y * y;
         }

     </script>
  </head>
  <body>

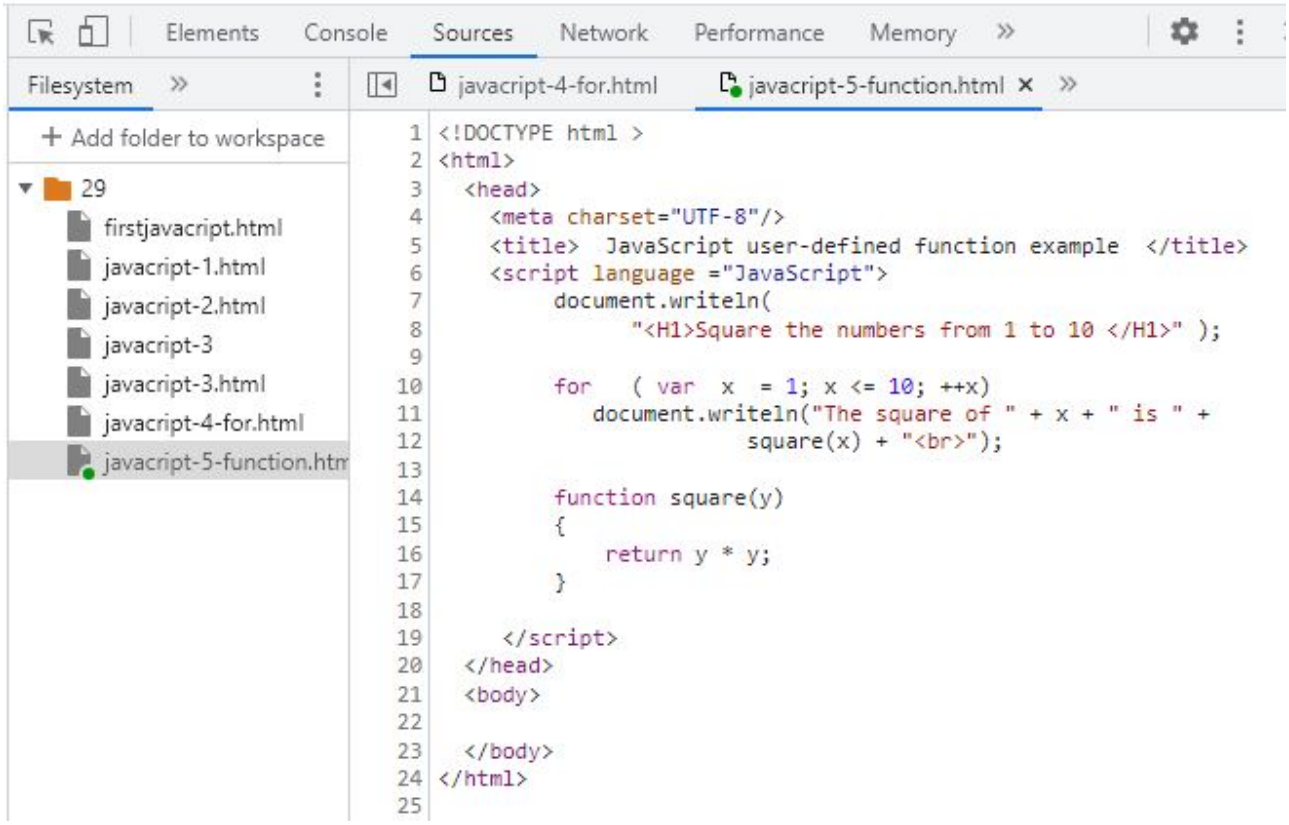
  </body>
</html>
```

After saving the html file and openng it by using a browser, you'll get:



Square the numbers from 1 to 10

The square of 1 is 1
The square of 2 is 4
The square of 3 is 9
The square of 4 is 16
The square of 5 is 25
The square of 6 is 36
The square of 7 is 49
The square of 8 is 64
The square of 9 is 81
The square of 10 is 100



In the above codes, line 10, 11 and 12 provides a for structure. Each iteration of the loop calculates the square of the current value of control variable x and outputs the result by writing a line in the HTML document. Function square is called on line 12 in the for structure with the expression:

square (x)

When function square (x) is called, a copy of the value of x is made automatically by the program and program control transfers to the first line of function square. Function square receives the copy of the value of x in the parameter y. Then square calculates y * y . The result is passed back to the point in line 12 where square was invoked.

Project 4. Answering questions (Please do this at your home by using your own computer)

1. What is data abstraction? Give one or two examples of data abstraction.

2. Why do we need to derive data from raw data?

Project 5. Creating a Professional Report

Summarize the above experiments procedure, results, answering questions and screenshots (project 1, 2, 3, 4) into one report. Your report is the assignment that is required to be submitted for evaluation on week 4. Create a report by following below steps.

Please note that

- as you may have a different version of Microsoft Word, the **following steps are just for your references.**

You can add a chapter called Chapter 3 in your previous report.

1. Open your last week's report and find the end of last week's report.
2. Copy this week's related experimental results, your findings and Screenshots, and paste them at the end of last week's report.
3. Delete original Table of Content you created.
4. Select all content, align all text to both left and right margin
5. Use shortcut key approach, generate Chapter 3: **JavaScript**
6. Then use shortcut key approach to generate proper sub-chapters for this week's lab work.
7. Insert Table of Contents to your report.